Question 1: **(a)** Many modern languages, including C++ and Java, allow two types of comments: in one case delimiters are used on *both ends* (e.g., /* and */); in the other, a delimiter marks only the *start* of the comment (e.g., //), which ends with the line. Explain briefly whether each of these is good or bad with respect to the criteria of *readability*, *writability*, and *reliability* – and *why*. (6 marks)

	Readability	Writability	Reliability
At both ends e.g. /* */	It is good since you can comment out an entire code segment that won't make it look too cluttered. It is also better at adding comments since you need only one at the beginning and end to indicate the commented section. No need for long one-lined comments.	It is a bit better. The reason is because a programmer only needs to add one beginning and ending delimiter (less to write and more organized). Defeats the purpose if it was inline.	Worse since it is more prone to an error. The ending delimiter could be forgotten, and it would comment out all the code that is further below.
At start only e.g. // //	It's a bit worse since it can make the program more difficult to read and more cluttered. If it is a long comment, then you would have to add more comment lines since you don't want it to be too long (keeps going to the right). It is good when using it as an inline comment, as it specifies the exactly which code it is referring to.	Worse. A programmer would have to constantly add new comments if the comment is long. Good for inline and only writing the delimiter at the beginning.	Better in Reliability as the programmer does not have to end the comment section (automatically ends); the comment ends the moment the programmer goes to a new line.

(b) Indicate for each of the *costs* of programming languages below whether it has generally *increased or decreased* over the last decades, and *why*. (4 marks)

Training:

Coding:

Tools:

Execution:

Reliability:

Question 2: **(a)** Using the BNF grammar below, draw a complete *parse tree* to show that the sentence `THE MAN PETS A DOG` *is* valid in this language. (3 marks)

```
<sentence>  -> <subject> <predicate>
<subject>   -> <article> <noun>
<predicate> -> <verb> <object>
<object>    -> <article> <noun>

<article> -> A | AN | THE
<noun>    -> CAT | DOG | MAN
<verb>    -> BITES | PETS
```

(b) Prove that the sentence `A DOG BITES PETS` is *not* in this language. (4 marks)

(c) Explain briefly how in BNF we could exclude sentences that *are* actually in the language, but should *not* be, such as `A MAN BITES THE DOG`. (3 marks)

(d) Is the above grammar *ambiguous?* Justify clearly your answer. (2 marks)

Question 3: **(a)** The BNF grammar on the right defines sentences comprising one or more X followed by one or more Y and one or more Z. What we really want, however, are sentences with the *same number* of X, Y, and Z i.e., of the form $X^N Y^N Z^N$ for N>0.

```
<S>  -> <Xs> <Ys> <Zs>
<Xs> -> X | <Xs> X
<Ys> -> Y | <Ys> Y
<Zs> -> Z | <Zs> Z
```

(a) Extend this grammar accordingly with the necessary *attributes, semantic rules,* and *predicates*. Explain your design clearly. (6 marks)

(b) Draw the parse tree for the sentence $X^2Y^3Z^2$ and *decorate* it clearly to show all synthetized attributes and their values, assuming *none are inherited.* Point out exactly *where* and *how* a syntax error is discovered. (4 marks)

Question 4: **(a)** What is a *raw string* in Python and what exactly is its purpose? Explain clearly using an appropriate example of your choosing. (3 marks)

(b) Explain *how* the output of the Python expression `print(j+2j)` can be: `4j`. Indicate which objects are created, in order. (3 marks)

(c) Explain in detail what the following Python statement does exactly and what its output is: `'Is Halloween important?'[-3::-13]` (3 marks)

Output: 'no'
It starts from index -3 which means it starts at the end of the string and goes through the entire string but jumps -13 steps.

(d) Indicate what the expression `lst[::2] = ['_'] * (len(lst)//2)` will do to a given list `lst`. Justify briefly your answer. (3 marks)

Question 5: **(a)** State the *differences* between the expressions `x,y=[y,x]` and `x,y=y,x` in terms of the objects created, the steps, and/or the result achieved. (2 marks)

(b) Indicate, for each of the statements below about Python dictionaries, whether it is *true or false*, and *why* exactly. (6 marks)

- Dictionary elements are accessed by key.

- Dictionaries are immutable data structures.

- Dictionaries can be nested to any depth.

- Dictionary keys can be anything.

(c) State which data structure a Python *set* uses internally, and *why*. (2 marks)

Question 6: **(a)** Consider the Python code below. Explain briefly but precisely what will happen to each of the two strings, and what the function call will do. (4 marks)

```
def fun():
    "hello, world!"
    "bye for now."
    print(fun)
fun()
```

(b) Write a *recursive* `minmax` function in Python that returns *both* the smallest and largest items from a given iterable. Make sure your code works in *all* cases, that it is *efficient* (traversing the iterable only once), and that it fully uses the language features (but not the `min`/`max` functions, of course). (8 marks)

Question 7: **(a)** Explain what the *diamond* problem is in object-oriented programming, why it *must* exist in Python, and *how* the language solves the issue. *(4 marks)*

(b) Explain the output of the `print` statement below, esp. what is compared and why. Define the `Comparable` abstract class that will act as an interface, then modify the `Point` class accordingly to "implement" this interface. Note that the same `print` statement should now show the correct result. (6 marks)

```
class Point:
    def __init__(self, x=0, y=0): self.x,self.y = x,y
    def __str__(self): return f'[{self.x}, {self.y}]'

print(Point(1,2) == Point(1,2))
```

```
class Comparable(
    '''a Java-like "interface" for comparing objects'''
```

```
class Point(Comparable):
    def __init__(self, x=0, y=0): self.x,self.y = x,y
    def __str__(self): return f'[{self.x}, {self.y}]'

    def
```

Question 8: **(a)** Write a single Python expression, using *only* `map/filter/reduce` with appropriate lambda expressions, to retrieve a hidden password by combining the second last letter of each N-letter long word in a given list of strings. (4 marks)

Example:
```
N=5; words=['mango', 'pear', 'lemon', 'nut', 'cherry',
            'pecan', 'orange', 'apple']   ->  'goal'
```

(b) Rewrite the expression below using *comprehension syntax* and simplifying as much as possible. Point out where a generator is used, if any. (3 marks)

```
reduce(operator.add (filter(lambda n: n>0, map(float,lst)))
```

(c) State *which* well-known Python higher-order function the `mystery` function below implements. Explain briefly *when* it is needed and *how* it works, using a simple example of your choosing. (5 marks)

```
def mystery(function, iterable):
    for it in iterable:
        yield function(*it)
```

Question 9: **(a)** Explain briefly what a *lazy data structure* is, and indicate *two* of its key benefits to a programming language. (3 marks)

(b) Answer the following about the `Polygon` class and code below. (9 marks)

```
class Polygon:  # comprises a list of 2D points: (X, Y, NAME)
    def __init__(self, *points): self._pts = points
    def __len__(self): return len(self._pts)
    def __str__(self): return ''.join(f'{name}({x},{y}) '
                                      for x,y,name in self._pts)

poly = Polygon((0,0,'O'), (1,2,'A'), (3,1,'B'))
```

(i) Will the following expression work? `for p in poly._pts: print(p)`
Explain exactly why, or not. If needed, modify the class to make it work.

(ii) Will the following expression work? `for p in poly: print(p)`
Explain exactly why, or not. If needed, modify the class to make it work.

(iii) Show (only) the *changes* to the class needed for the following to work.

```
for _ in range(5): print( next(poly), end=' ')  #->
(0, 0, 'O') (1, 2, 'A') (3, 1, 'B') None (0, 0, 'O')
```

Question 5: **(a)** Explain in detail the Python expression `x,y=[1,2j]`. You should describe all the steps clearly, in order, referring as needed to e.g., types, references, bindings, etc. Also, point out all the features that are unique to Python. (7 marks)

(b) Write a Python expression that uses *slicing* to produce `'13578642'` from the given string `s='12345678'`, or `'acdb'` if the string is `s='abcd'`, etc. (Assume that the input string always has an even length.) Point out what the equivalent code in C++ would be like, and hence the *benefits* of slicing syntax. (3 marks)

Question 4: **(a)** Complete the rules of *denotational semantics* hereafter to define Boolean operators according to the given BNF grammar. (5 marks)

```
<B>   ->   True
<B>   ->   False
<B>   ->   ( <B> )
<B>   ->   not <B>
<B>   ->   <B>  and  <B>
<B>   ->   <B>   or  <B>
```

Denotational semantics:

```
Me(<B>)  =  1
Me(<B>)  =  0

Me(<B>
```

(b) Apply your denotational semantics in part (a) to *evaluate* the meaning/value of the expression `((A or B) and (not C))`, assuming `A` and `C` are `False` and `B` is `True`. Show *all* the steps of your calculation. (2 marks)

Question: 3 (a) Describe what data structure the BNF grammar rules below define, what the format is exactly. Give one example of the simplest possible sentence and one example of a sentence where both recursive cases are applied twice. (4 marks)

```
<S> -> [ <L> ]
<L> -> <I> | <L> , <I>
<I> -> <S> | <N>
<N> -> 1 | 2 | 3 | 4
```

(b) Show which rule/s of the grammar in part (a) can be written better in EBNF. Explain accordingly whether EBNF is *useful,* or not, in this case. (2 marks)

(c) Using the BNF grammar below, write a detailed *left-most derivation* to show that the sentence `'the cat likes to eat and play'` is valid. (3 marks)

```
<s>  -> <np> <vp>
<np> -> <d> <n>
<vp> -> <av> <v>
<vp> -> <av> <v> and <v>

<d>  -> a | the | every
<n>  -> cat | dog | hamster
<av> -> can | 'likes to'
<v>  -> eat | play | run
```

Question 2: (a) Does *garbage collection* (GC) increase or decrease the various *costs* of a programming language listed below? *Justify* briefly your answers. (6 marks)

Training:

Coding:

Compilation:

Execution:

Tools:

Reliability:

(b) List out all the *steps* of the compilation process that starts with a Java source code file and ends with a running app. (No need for details.) Indicate which steps are optional, if any. (3 marks)

Question 1: **(a)** Modern programming languages distinguish *uppercase* from *lowercase* in identifiers (e.g., variable and function names). Elaborate the reasons for this design decision in terms of *readability*, *writability*, and *reliability*. (6 marks)

(b) With reference to programming language characteristics (evaluation criteria), indicate precisely *why* Python offers *only* the `list` class as linear container whereas C++ has many e.g., `vector`, `array`, `list`, `fwd_list`... (3 marks)

(d) The BNF grammar rules below define C++ output statements, where `<ostream>` can be any output stream variable e.g., `cout`, also `<string>` represents any valid string object, and `<int>` denotes a literal integer.

```
<xpr> -> <xpr> '<<' <xpr> | <obj>
<obj> -> <ostream> | <string> | <int>
```

(i) Explain from principles exactly *why* this grammar is *ambiguous.* (3 marks)

(ii) To prove the above, show two different *parse trees* for the sentence:
`cout << "n=" << 8` (3 marks)

Question 1: **(a)** Write a *single* Python *expression* for each task *below.* *(6 marks)*

– Retrieve the `n` *largest* items from a tuple `t` of numbers
e.g., `n,t = 3,[2,6,11,3,17,0,8,13,5,10] -> [11,13,17]`

– Check whether a dictionary `d` has duplicate values or not (true or false)

– Given a list `ls`, print the tuples that consist of each list element and its position i.e., first element with number 1, etc. exactly as shown:
e.g., `ls=['a','b','c'] -> ('a', 1) ('b', 2) ('c', 3)`

(b) Write a Python script that rolls twenty dice and stores the frequency of each throw in a dictionary. Use `random.randrange()`, which has the same syntax as `range()`, to generate dice throws. Lastly, print the results formatted exactly as per the example shown below. Make sure your code is Pythonic. (6 marks)

```
1 : ****
2 : **
3 : ***
4 : ****
5 : *****
6 : **
```

Question 2: **(a)** Explain one feature of *Boolean expressions* in Python which does not exist in C++ or Java. Illustrate the benefit with a well-chosen example. (3 marks)

(b) Consider the Python code below. What are the lists `l1` and `l2` if the output is `[(1,2,3), ('a','b','c')]` ? Explain briefly how this works. (3 marks)

```
z=zip(l1,l2)
print(list(zip(*z)))
```

Question 3: **(a)** Contrary to C++ and Java, Python *functions* can return *multiple values*. Is that statement correct? Explain in detail using an example. (3 marks)

(b) Write *two* Python *functions* that mimic the C++ functions below as closely as possible. Explain what will happen exactly when the interpreter will evaluate your code. What does this example demonstrate clearly? (4 marks)

```
int fun( int a ) { return 1 + a; }
int fun( int a, int b ) { return 1 + a*b; }
```

(c) Give then explain the output of the two Python expressions `len([1,3,5])` and `len(135)`. Explain accordingly what the `len` function really is. (3 marks)

Question 4: (a) Implement a Python class called `T`, exactly as specified hereafter. Make sure to use standard OOP and Python coding and naming practices. (8 marks)

- Class `T` has a "private" *member variable* to store the instance's name, which must be a user-provided string, or else a `TypeError` exception will be raised.
- Class `T` also has a "private" *class variable* to store the names of all current and past instances of the class. Each name can be used only once. If one attempts to reuse a previous name, a `ValueError` exception will be raised.
- Class `T` implements a function to allow printing a class instance, exactly as shown below. The function should work just the same even if subclassing.
- Class `T` has a function that returns all current and past names, as shown below.

```
t=T('one') ; print(t)    ->   T instance with name "one"
print(T('two'))          ->   T instance with name "two"
print(T.names())         ->   ['one', 'two']
more = T('one')          ->   ValueError: name already used
```

(b) Define X, a minimal subclass of T that adds a "public" size attribute. (4 marks)

Question 5: **(a)** Explain *each step* of the Python expression below, *in order*, stating what it does exactly and showing what data it produces. (5 marks)

```
list(map(lambda t: str(Fraction(*t)), enumerate(range(1,9,2))))
```

(b) For each task below, write a *single expression* using Python functions and higher-order functions (but *not* comprehensions) to yield the result. (5 marks)

– Give the number of family members older than a given age threshold `t`

e.g., `t=21, family = ({'name': 'Alice', 'age': 23},`
`{'name': 'Bob', 'age': 27},`
`{'name': 'Charlie', 'age': 18})  ->  2`

– Add all the bits of the binary representation of an integer `N` (using the `bin` function e.g.: `bin(5)` returns `'0b101'`, `bin(13)` returns `'0b1101'`, etc.)

e.g., `N=5 -> 2,  N=13 -> 3,  N=127 -> 7`

(b) For each task below, write a *single expression* using Python functions and higher-order functions (but *not* comprehensions) to yield the result. (5 marks)

– Give the number of family members older than a given age threshold `t`

e.g., `t=21, family = ({'name': 'Alice', 'age': 23},`
`{'name': 'Bob', 'age': 27},`
`{'name': 'Charlie', 'age': 18})  ->  2`

– Add all the bits of the binary representation of an integer `N` (using the `bin` function e.g.: `bin(5)` returns `'0b101'`, `bin(13)` returns `'0b1101'`, etc.)

e.g., `N=5 -> 2,  N=13 -> 3,  N=127 -> 7`

END OF PAPER

```
--(self):
    return self.name
```

Question 1: **(a)** Implement a Python class called `T`, exactly as specified hereafter. Make sure to use standard Python coding and naming practices. (7 marks)

3.5

- Class `T` has a "private" *member variable* to store the instance's name, which must be a user-provided string, or else a `TypeError` exception will be raised.
- Class `T` also has a "private" *class variable* to store the names of all current and past instances of the class. Each name can be used only once. If one attempts to reuse a previous name, a `ValueError` exception will be raised.
- Class `T` implements a function to allow printing a class instance, exactly as shown below. The function should work just the same even if subclassing.
- Class `T` has a function that returns all current and past names, as shown below.

```
t=T('one') ; print(t)    ->   T instance of name "one"
print(T('two'))          ->   T instance of name "two"
print(T.names())         ->   ['one', 'two']
```

```
all_names = []

__init__(self, n):
    if n in T.?
        raise ValueError("Name exists!")
    elif isinstance(n, str):
        T.name = n
        all_names.append(n)
    else:
        raise TypeError("Must be a string")

__str__ ?!

names(self):
    print(all_names)

__len__(self)
    return self.name
```

(b) Define `X`, a minimal subclass of `T` that adds a "public" size attribute. (3 marks)

0.5

```
class X:
    size = 1      # int value
    """
```

Question 3: (a) Give the *output* of the sample for loop below. Explain clearly how the code works i.e, give *all the key steps*, from the for loop itself to the methods of class S (as and when they are called). What does S do then? (7 marks)

```
class S:
    def __init__(self, data, N):
        self.data, self.index, self.step = data, 0, N
    def __iter__(self): return self
    def __next__(self):
        if self.index >= len(self.data): raise StopIteration
        self.index += self.step
        return self.data[self.index - self.step]

for item in S(range(2,24), 5) : print(item, end=' ')
```

range(2,24) will generate values from 2 to 23 NOT true

The generated values will take the data parameter.

index set to 0.

step set to 5.

This is an iterable container. It stops iterating when the index becomes greater than or equal to data.

The index increments by following the passed step function.

The one element from .data is returned. The index of this element is computed by self.index - self.step

answer true

(b) Implement the repeat *generator*. Two typical use cases are shown below. Recall that the second argument to repeat, if provided, is the number of repetitions; without it, the generator is potentially infinite. (5 marks)

```
list(repeat('@', 7)) -> ['@', '@', '@', '@', '@', '@', '@']
set(map(pow, range(1,6), repeat(2))) -> {1, 4, 9, 16, 25}
```

Question 4: (a) Describe precisely but concisely what the following BNF grammar rule generates, using simple, natural English (not programming jargon...) (2 marks)

```
<S> -> 0 <S> 1 | 01
```

The token <S> can give out '0 <S> 1' or 01.
The <S> in '0<S>1' can again expand to 00<S>11 and this can go on until it expands to 01 which is terminal.
This token is used to genate a combination of 0's and 1's not only!

(b) Using the BNF grammar below, give a detailed left-most *derivation* for the sentence aaccdcc. (3 marks)

```
<S> -> a <S> c <B>
<S> -> <A> | b
<A> -> c <A> | c
<B> -> d | <A>
```

<S> → a <S> c <B>
→ a a <S> c <B> c <B>
→ a a <A> c <B> c <B>
→ a a c c <B> c <B>
→ a a c c d c <B>
→ a a c c d c <A>
→ a a c c d c c

(c) Show a reverse derivation to prove that the sentence acdc cannot be produced by the grammar in part (b). Explain the error precisely. (3 marks)

acdc
→ a c d <A>
→ a c <B> <A>
→ ?

Left most ✓

The first rule <S> → a <S> c <B>
is 4 characters same as acdc.
It is the only rule which gives 'a'.
However, 'a' is directly followed by <S>.
Now <S> can evaluate to <A> then c but
then we have 'acc<B>' then 'accd'
Not possible to get 'acd' to begin with.

stion 1: (a) Write the code of the following three Python classes, exactly as specified. Make sure to use standard Python coding and naming practices. (10 marks)

- The abstract Bird class has a member variable to store a bird's name, which users must provide when creating a new bird. It has a class variable to store the sound every bird makes, which is "chip" by default. It implements a method to allow printing a bird's details, as per the examples below. The same method must be used by all concrete subclasses (and it cannot be overridden).

```
print(Dove('Love'))    ->    Love the Dove says "cool"
print(Duck('Huck'))    ->    Huck the Duck says "kwak"
```

- The Dove class is a concrete subclass of Bird that has an additional age attribute, with a default value of 0. It changes the sound every dove makes to "cool".

- The Duck class is a concrete subclass of Bird that changes the sound every duck makes to "kwak". It does not have any additional attribute.

(b) Explain how the class *comparison operator* in Java vs. in Python are similar or different. Consider how/where they are defined, how they work... (4 marks)

Class comparison in Java and in Python are both used to compare between 2 values. But there are some differences in Java its represented as '==' and in Java it checks if the objects refer to the same memory location. in Python there are many comparison operators such as '==' for values a 'is' for references and checks in the memory location sim to Java. Both support comparison but syntax and beh vari

Question 2: **(a)** Explain each step of the Python expression below (starting with the range) i.e., state briefly for each what it does and show what data it produces. Lastly, indicate how to modify the code minimally so as to add the terms together instead, producing a single number as result. (5 marks)

```
list( map(lambda t: str(Fraction(*t)), enumerate(range(1,11,2), 1)))
```

(b) For each task below, write a single expression using Python functions, higher-order functions, and/or comprehensions, to yield the desired result. (5 marks)

– Calculate the average length of all iterables in a dictionary `d`
e.g., `d = {'A':[1,2,3], 'B':{4,5}, 'C':(6,7,8,9)} -> 3.0`

```
sum ( map(len (d.values())) / len (d)
```

– Calculate the sum of every other digits in a given integer `N`

e.g., `N = 1234567890 -> 25` # because 1+3+5+7+9 = 25
`N = 73451 -> 12` # because 7+4+1 = 12

```
sum (
```

(c) Explain what exactly is returned by the function below and what, accordingly the print statement outputs, or which error is produced, if any. Explain how this relates to the use of global variables. (4 marks)

```
def expB(base): return lambda exp: base**exp
e2=expB(2) ; del expB; print( e2(8) )
```

(b) For each task below, write a single expression using Python functions, higher-order functions, and/or comprehensions, to yield the desired result. (5 marks)

– Calculate the average length of all iterables in a dictionary `d`

e.g., `d = {'A':[1,2,3], 'B':{4,5}, 'C':(6,7,8,9)} -> 3.0`

```
sum(map(len(d.values())))/len(d)
```

– Calculate the sum of every other digits in a given integer `N`

e.g., `N = 1234567890 -> 25` # because 1+3+5+7+9 = 25

`N = 73451 -> 12` # because 7+4+1 = 12

```
sum(
```

by the function below and what, accordiv

Question 3: (a) Examine the function below and give the output of the two function calls. Explain briefly how this function works, justifying the use of `iter` and `try/except` in the code. Which well-known function does it implement? (7 marks)

```
def mystery(function, iterable, initializer=None):
    it = iter(iterable)
    if initializer is None:
        try: value = next(it)
        except StopIteration: return
    else: value = initializer
    for element in it: value = function(value, element)
    return value

mystery(operator.add, [1,2,3,4,5])
mystery(operator.mul, [7,2,4], 10)
```

....e the function below and give the output of the two function calls.
....n briefly how this function works, justifying the use of iter and try/except in the code. Which well-known function does it implement? (7 marks)

```
def mystery(function, iterable, initializer=None):
    it = iter(iterable)
    if initializer is None:
        try: value = next(it)
        except StopIteration: return
    else: value = initializer
    for element in it: value = function(value, element)
    return value

mystery(operator.add, [1,2,3,4,5])
mystery(operator.mul, [7,2,4], 10)
```

(b) Implement map as a generator function, first (i) assuming that a single iterab is passed as argument, then (ii) in the general case, as shown below. (5 mark

```
list(map(square, [1,2,3,4,5]))    -> 1, then 4, 9, 16, and 2
list(map(pow, [2,3,5], [5,3,2])) -> 32.0, then 27.0, and 25
```

(i)
```
def map(function, iterable):
```

(ii)
```
def map(
```

(c) Draw the *decorated parse tree* for the character string `4Hbad`, assuming that all attributes are synthetized. Show where the syntax error is found. (3 marks)

END OF PAPER

Question 1: (a) Explain clearly but concisely how the three expressions below compare in terms of *readability*, *writability*, and *reliability*. (5 marks)

`x//y` *vs.* `Math.floor(x/y)` *vs.* `int(x/y)`

(b) Consider the five *cost criteria* for programming languages that are: training, coding, tools, execution, and reliability. Explain briefly for each whether it has generally increased or decreased over the last decades, and why. (5 marks)

Training:

Coding:

Tools:

Execution:

Reliability:

Question 2: **(a)** What is a *raw string* in Python and what exactly is its purpose? Explain clearly using an appropriate example of your choosing. (3 marks)

(b) Explain what the expression `x='Y'` does in Python vs. the same in C++ (state your assumptions, if any). Point out what design decision/s this example illustrates, then give one advantage and one drawback for each. (4 marks)

(c) Write a *single* Python expression using *slicing* for each case below. Note that your expression should work regardless of the string length. (5 marks)

- before: `s='ttAxxByyCzz'` -> after: `s='ABCxyz'`

- before: `l=[0, 2, 0, 4, 0, 6]` -> after: `l=[1, 2, 3, 4, 5, 6]`

Question 3: **(a)** Write a *recursive* `minmax` function in Python that returns *both* the smallest and largest items from a given iterable. Make sure your code works in *all* cases, and uses the language features (but not `min` or `max`, of course). (6 marks)

(b) Indicate what `fun(N)` calculates, given the Python code below. Explain briefly the code logic, and especially what each step of `reduce` does exactly. Use the example of the call `fun(6)` to illustrate your answer in detail. (6 marks)

```
fun = lambda n: reduce(lambda x,n: (x[1], x[0]+x[1]),
                       range(n), (0,1)) [0]
```

Question 4: **(a)** Explain briefly but precisely *what each line* of the Python script below *does*, and give the *output*, or the error, if any. (4 marks)

```
class alist: pass                          #1
lst = alist()                              #2
lst.__doc__ = 'the wiser'                  #3
print(alist.__doc__, lst.__doc__)          #4
```

(b) Explain the output of the `print` statement below, esp. what is compared and why. Define the `Comparable` abstract class that will act like a Java interface, then modify the `Point` class accordingly to "implement" this interface. Note that the same `print` statement should now show the correct result. (6 marks)

```
class Point:
    def __init__(self, x=0, y=0): self.x,self.y = x,y
    def __str__(self): return f'[{self.x}, {self.y}]'

print(Point(1,2) == Point(1,2))
```

```
class Comparable(
```

```
class Point(Comparable):
    def __init__(self, x=0, y=0): self.x,self.y = x,y
    def __str__(self): return f'[{self.x}, {self.y}]'

    def
```

Question 4: **(a)** Explain briefly but precisely *what each line* of the Python script below *does*, and give the *output*, or the error, if any. (4 marks)

```
class alist: pass                                    #1
lst = alist()                                        #2
lst.__doc__ = 'the wiser'                            #3
print(alist.__doc__, lst.__doc__)                    #4
```

(b) Explain the output of the `print` statement below, esp. what is compared and why. Define the `Comparable` abstract class that will act like a Java interface, then modify the `Point` class accordingly to "implement" this interface. Note that the same `print` statement should now show the correct result. (6 marks)

```
class Point:
    def __init__(self, x=0, y=0): self.x,self.y = x,y
    def __str__(self): return f'[{self.x}, {self.y}]'

print(Point(1,2) == Point(1,2))
```

```
class Comparable(
```

```
class Point(Comparable):
    def __init__(self, x=0, y=0): self.x,self.y = x,y
    def __str__(self): return f'[{self.x}, {self.y}]'

    def
```

Question 4 (a) Explain briefly but precisely what each line of the Python script below does, and give the output, or the error, if any. (4 marks)

```
class slist: pass
lst = slist()
lst.__doc__ = 'the wiser'
print(slist.__doc__, lst.__doc__)
```

(b) Explain the output of the `print` statement below, esp what is compared and why. Define the `Comparable` abstract class that will act like a Java interface, then modify the `Point` class accordingly to "implement" this interface. Note that the same `print` statement should now show the correct result. (6 marks)

```
class Point:
    def __init__(self, x=0, y=0): self.x,self.y = x,y
    def __str__(self): return f'[{self.x}, {self.y}]'

print(Point(1,2) == Point(1,2))
```

```
class Comparable(
```

```
class Point(Comparable):
    def __init__(self, x=0, y=0): self.x,self.y = x,y
    def __str__(self): return f'[{self.x}, {self.y}]'

    def
```

Question 5: (a) Write a single Python expression, using *only* `map/filter/reduce` with appropriate lambda expressions, to retrieve a hidden password by combining the last letter of each N-letter long word in a given list of strings. (4 marks)

Example: `N=5; words=['mango', 'pear', 'lemon', 'nut', 'cherry', 'orange', 'apple', 'banana']` -> `'one'`

(b) Rewrite your expression in part (a) using *comprehension syntax* as much as possible, and one string function. Explain why you need the latter. (3 marks)

(c) State *which* well-known Python higher-order function the `mystery` function below implements. Explain briefly *when* it is needed and *how* it works, using a simple example of your choosing. (5 marks)

```
def mystery(function, iterable):
    for it in iterable:
        yield function(*it)
```

...stion 6: (a) Point out what is *similar* and what is *different* between the two lines of code below given that e.g., `alist= range(1_000_000)`. (3 marks)

```
for item in alist[::-1]: print(item)
for item in reversed(alist): print(item)
```

(b) Implement `reversed` as an *Iterator* so the above code works fine. (5 marks)

```
class reversed:
```

(c) Now implement `reversed` as a simple *generator* function. (4 marks)

```
def reversed(data):
```

Question 6: (a) Point out what is *similar* and what is *different* between the two lines of code below given that e.g., `alist= range(1_000_000)`. (3 marks)

```
for item in alist[::-1]: print(item)
for item in reversed(alist): print(item)
```

(b) Implement `reversed` as an *iterator* so the above code works fine. (5 marks)

```
class reversed:
```

(c) Now implement `reversed` as a simple *generator* function. (4 marks)

```
def reversed(data):
```

Question 8: (a) Using the grammar below, draw a *parse tree* to show that the sentence THE MAN PETS A DOG *is* in the language. (3 marks)

```
<sentence>  -> <subject> <predicate>  |
<subject>   -> <article> <noun>       | <article> -> A | AN | THE
<predicate> -> <verb> <object>        | <noun>    -> CAT | DOG | MAN
<object>    -> <article> <noun>       | <verb>    -> BITES | PETS
```

(b) Prove that the sentence DOG BITES MAN is *not* in the language. (2 marks)

(c) The sentence A MAN BITES THE DOG *is* also in the language, but we don't want that. How can we extend the grammar to exclude it? Explain briefly. (3 marks)

(d) Is the above grammar *ambiguous*? Justify your answer. (2 marks)

Question 8: **(a)** Using the grammar below, draw a *parse tree* to show that the sentence THE MAN PETS A DOG *is* in the language. (3 marks)

```
<sentence>  -> <subject> <predicate>   |
<subject>   -> <article> <noun>        | <article> -> A | AN | THE
<predicate> -> <verb> <object>         | <noun>    -> CAT | DOG | MAN
<object>    -> <article> <noun>        | <verb>    -> BITES | PETS
```

(b) Prove that the sentence DOG BITES MAN is *not* in the language. (2 marks)

(c) The sentence A MAN BITES THE DOG *is* also in the language, but we don't want that. How can we extend the grammar to exclude it? Explain briefly. (3 marks)

(d) Is the above grammar *ambiguous*? Justify your answer. (2 marks)

Question 9: Character strings in Fortran I were expressed in the following format, where `<number>` is a positive integer, `H` is a literal key, and `<string>` is a sequence of one or more letters: `<string-literal> -> <number> H <string>`

Furthermore, a string literal is correctly defined *only* if the value of `<number>` matches the length of `<string>`. For instance, `5Hhello` and `3Hbye` are valid character strings in Fortran I, whereas `2Hbad` and `10Hworse` are not.

(a) Indicate what kind of problems *attribute grammars* are solving, then point out the specific issue in the case of Fortran I strings. (3 marks)

(b) Give the necessary *attributes, predicates,* and *semantic rules* (incl. look-up) of an attribute grammar for character string literals in Fortran I. (6 marks)

(c) Draw the *decorated parse tree* for the character string `4Hbad`, assuming that all attributes are synthetized. Show where the syntax error is found. (3 marks)

END OF PAPER

Question 7: (a) Describe, in simple English (not programming lingo), exactly *what kind of sentences* match the regular expression: `"(0[xX])?[0-9a-fA-F]+"` (2 marks)

(b) Explain the two regular expressions `"\.(.*)\."` vs. `"\.(.*?)\."` and how they match an IP such as `"198.111.345.001"`. Show the results. (4 marks)

(c) Give the two *regular expressions* `m` and `s` such that `re.sub(m,s,text)` will replace all *double* occurrences of a word in the given `text` string by a *single* occurrence of that word. Hint: use backward references. (4 marks)

Example: `'to be be or not to be, that is the the question'`
→ `'to be or not to be, that is the question'`